

LLD Practice Platform

A Low-Level Design (LLD) practice platform providing structured, evidence-based rubric feedback and progressive attempt tracking.

Brief

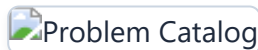
Built as a 2-day engineering assignment for **CipherSchools**, this platform provides a focused practice environment for Low-Level Design interviews. Learners select a canonical design problem (Parking Lot, Elevator System, Vending Machine), write a plain-text/pseudocode solution detailing domain classes, interfaces, responsibilities, relationships, and design trade-offs, and receive structured feedback scored across fixed rubric dimensions.

Every attempt is persisted and scored with verbatim evidence citations from the submission. Learners can retry problems and inspect a **delta view** showing per-dimension score progressions across attempts.

Screenshots & User Journey

1. Problem Catalog

Select from canonical Low-Level Design challenges with defined requirements and constraints.



Problem Catalog

2. Practice Studio & Monospace Editor

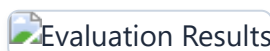
Dual-panel workspace with problem requirements and rubric drawer on the left, and pseudocode editor with live structural indicators on the right.



Practice Studio

3. Structured Rubric Feedback & Evidence Verification

Comprehensive evaluation featuring overall score, executive summary, and verified verbatim quotes from the learner's text.



Evaluation Results

4. Attempt History & Progressive Learning Deltas

Track iterative skill improvements across retries with per-criterion score diffs.



Attempt History Deltas

Features

Problem Selection: Choose from 3 seeded canonical LLD challenges (**Parking Lot**, **Elevator System**, and **Vending Machine**) complete with functional requirements, constraints, and tags.

Text & Pseudocode Submission: Distraction-free monospace editor focused on software design thinking (class contracts, encapsulation, abstractions) without the friction of visual diagram tools.

Deterministic Validation Checks: Validates input length (minimum 100 characters) and requires at least 2 structural OOP tokens (`class` , `interface` , or PascalCase entity names). Short-circuits trivial, empty, or junk input immediately without wasting AI API credits.

Content Hashing & Idempotency: Computes a SHA-256 hash of the trimmed submission. If an identical design has already been evaluated for that problem, the existing evaluation is reused instantly.

Structured AI Evaluation: Uses the Anthropic Claude API with strict system-prompt JSON schema enforcement to evaluate submissions across 7 core design dimensions.

Evidence Verification Guard: A deterministic post-check that verifies AI-quoted evidence strings actually appear in the learner's submission text. Mismatches flag `evidenceVerified: false` and force `confidence: 'low'` (surfaced as "Needs Review" in the UI) without crashing or discarding the evaluation.

Attempt History & Delta View: Chronological attempt log displaying past submissions and calculating per-criterion deltas against the immediately preceding attempt (e.g. `Coupling/Cohesion: 2 → 4 (+2 ↑)` , `Extensibility: 2 ↔ unchanged`).

Tech Stack

Backend: Node.js + Express

Database: MongoDB + Mongoose

Frontend: React + Vite + Tailwind CSS

AI Integration: Anthropic Claude API (`@anthropic-ai/sdk`) with structured JSON output enforced via system prompt

Testing: Jest + Supertest, `mongodb-memory-server` / local MongoDB for integration tests

Infrastructure: Docker + Docker Compose (one application service, one MongoDB service)

Other: `crypto` (built-in Node.js module) for SHA-256 content hashing (idempotency), `dotenv` for environment variable configuration

Architecture / How It Works

2-Day MVP Prototype Architecture & Evaluation Flow

```
mermaid
flowchart TD
```

```

subgraph Client["Frontend Layer (React + Vite + Tailwind)"]
  UI_Catalog["Problem Catalog\n(Parking Lot, Elevator, Vending)"]
  UI_Editor["Monospace Studio\n(Plain Text / Pseudocode)"]
  UI_Polling["Simple Polling Loop\n(Every ~2s)"]
  UI_Results["Evaluation & Delta View\n(Scores, Evidence & Deltas)"]
end

subgraph API["Backend Monolith (Node.js + Express)"]
  Router["Express REST Router\n(/api/problems, /api/attempts, /api/submissions)"]
  Service_Sub["SubmissionService\n(State Machine & Immediate Persistence)"]
  Service_Delta["DeltaService\n(Per-Criterion Progress Comparison)"]
end

subgraph Pipeline["MVP Evaluation Pipeline"]
  Gate{"Deterministic Check\n(≥100 chars & ≥2 OOP tokens?)"}
  ShortCircuit{"Deterministic Short-Circuit\n(Score: 0, Instant Feedback)"}
  Idempotency{"SHA-256 Hash Check\n(Duplicate submission?)"}
  ReuseCache["Reuse Existing Evaluation\n(Avoids Re-running AI)"]
  AI_Call["AI Evaluator Engine\n(7 Rubric Dimensions & Exact Quotes)"]
  EvidenceGuard{"Evidence Verification Guard\n(Does quote appear in text?)"}
  Verified["evidenceVerified: true\nconfidence: 'high'"]
  Unverified["evidenceVerified: false\nconfidence: 'low' (Needs Review)"]
end

subgraph Persistence["Data & AI Services"]
  MongoDB["MongoDB Database\n- Problems\n- Attempts\n- Submissions\n- Evaluations"]
  AI_Service["AI Evaluator Provider\n(Anthropic Claude API or Built-in Local Fallback)"]
end

%% Flow Steps
UI_Editor -->|"1. POST /api/attempts/:id/submissions"| Router
Router --> Service_Sub
Service_Sub -->|"2. Save status = 'pending'"| MongoDB

Service_Sub --> Gate
Gate -- "Fails (Too short / No OOP)" --> ShortCircuit
ShortCircuit -->|"Status = 'completed'"| MongoDB

Gate -- "Passes" --> Idempotency
Idempotency -- "Duplicate Hash Found" --> ReuseCache
ReuseCache -->|"Reuse Evaluation"| MongoDB

Idempotency -- "Unique Content" --> AI_Call
AI_Call -->|"3. Send prompt with rubric"| AI_Service
AI_Service -->|"4. Structured JSON output"| AI_Call

AI_Call --> EvidenceGuard
EvidenceGuard -- "Quote Verified" --> Verified
EvidenceGuard -- "Quote Fabricated" --> Unverified

Verified -->|"5. Save Evaluation & Status = 'completed'"| MongoDB
Unverified -->|"5. Save Evaluation & Status = 'completed'"| MongoDB

UI_Polling -->|"6. GET /api/submissions/:id"| Router
Router -->|"7. Evaluation Result"| UI_Results
Service_Delta -->|"8. Compute per-criterion deltas"| UI_Results

```

Domain Model

Problem: Represents a design challenge (`slug`, `title`, `description`, `requirements[]`, `constraints[]`, `difficulty`, `tags[]`).

Attempt: Links a learner session to a problem (`id`, `problemId`, `userId`, `status` ('in_progress' | 'submitted'), `startedAt`). Separate from Submission so a draft or abandoned attempt does not trigger evaluation.

Submission: Owns *what was submitted* (`id`, `attemptId`, `problemId`, `userId`, `content`, `contentHash`, `submittedAt`, `evaluationStatus` ('pending' | 'evaluating' | 'completed' | 'failed')). Immutable once created.

Evaluation: Owns *judgement about the submission* (`id`, `submissionId`, `evaluatorType` ('deterministic' | 'ai'), `rubricResults[]`, `overallSummary`, `createdAt`). Each `RubricResult` contains: `criterion`, `score` (0–5), `evidence` (verbatim quote), `concern`, `suggestion`, `confidence` ('high' | 'low'), and `evidenceVerified` (boolean).

Project Structure

```
LLD/
├── client/                                # React + Vite + Tailwind CSS frontend
│   ├── src/
│   │   ├── api/client.js                # API wrapper for backend calls
│   │   ├── components/                  # MonospaceEditor, RubricCard, DeltaBadge, etc.
│   │   ├── pages/                      # ProblemListPage, PracticePage, AttemptHistoryPage
│   │   ├── App.jsx                     # Main routing & state
│   │   └── index.css                   # Tailwind directives & design tokens
│   └── package.json
├── server/                              # Node.js + Express backend
│   ├── src/
│   │   ├── config/                    # db.js, rubric.js (7 static rubric dimensions)
│   │   ├── controllers/               # problemController, attemptController, submissionController
│   │   ├── evaluators/                # Evaluator, DeterministicEvaluator, AIEvaluator,
│   │   │   EvidenceVerifier, EvaluatorService
│   │   ├── models/                   # Problem, Attempt, Submission, Evaluation
│   │   ├── routes/                   # Express route definitions
│   │   ├── scripts/                  # seed.js (seeds the 3 canonical problems)
│   │   └── services/                 # problemService, attemptService, submissionService,
│   │       deltaService
│   ├── app.js                         # Express application configuration
│   ├── server.js                      # Server listener endpoint
│   ├── tests/                        # Jest + Supertest test suites
│   ├── Dockerfile                    # Backend production container specification
│   └── package.json
├── docker-compose.yml                 # App container + MongoDB service definition
└── AI_USAGE.md                       # Detailed log of AI interactions & engineering decisions
```

```
├─ package.json
└─ README.md
```

Monorepo root scripts

Getting Started

Prerequisites

Node.js: v18 or higher (v20+ recommended)

MongoDB: A running MongoDB instance locally (`mongodb://127.0.0.1:27017`) or Docker

1. Environment Configuration

Create a `.env` file in the `server/` directory:

```
env
PORT=5000
MONGODB_URI=mongodb://127.0.0.1:27017/lld_practice
ANTHROPIC_API_KEY=your_anthropic_api_key_here
ANTHROPIC_MODEL=claude-sonnet-4-6
NODE_ENV=development
```

(Note: If `ANTHROPIC_API_KEY` is not provided, the platform automatically activates a local mock evaluator fallback that extracts real verbatim quotes from user code so you can test the full pipeline offline).

2. Running with Docker (Recommended)

To run both the application and MongoDB in containers:

```
bash
docker-compose up --build
```

3. Running Without Docker

Install dependencies:

```
bash
```

Root and workspace dependencies

```
npm run install:all
```

Or individually:

```
cd server && npm install
cd ../client && npm install
```

Seed problems into MongoDB:

```
bash
npm run seed
```

or: `cd server && node src/scripts/seed.js`

Start the development servers:

```
bash
```

Start both server and client concurrently

```
npm run dev
```

Or run in separate terminals:

```
npm run dev:server    # Backend API on http://localhost:5000
npm run dev:client    # Frontend on http://localhost:5180 (or http://localhost:3000)
```

API Endpoints

| Method | Route | Description | |---|---|---| | [GET](#) | [/api/problems](#) | List all available problems with requirement counts and tags | | [GET](#) | [/api/problems/:slug](#) | Retrieve single problem detail (requirements, constraints, rubric criteria) | | [POST](#) | [/api/problems/:slug/attempts](#) | Start a new attempt session for a problem (`userId` optional, defaults to `"demo-user"`) | | [POST](#) | [/api/attempts/:id/submissions](#) | Submit design text; immediately saves as `pending` and kicks off evaluation pipeline | | [GET](#) | [/api/submissions/:id](#) | Poll submission status (`pending` → `evaluating` → `completed` / `failed`) and view populated evaluation | | [GET](#) | [/api/problems/:slug/attempts?userId=](#) | Retrieve attempt history for a problem with per-criterion score deltas |

Testing

Run the automated backend test suite:

```
bash
npm run test:server
```

or: cd server && npm test

What Is Tested

Submission State Machine & Resilience: Asserts that a submission is saved and marked `failed` with an error message even if the AI evaluator throws (data is never lost).

Idempotency: Asserts that submitting identical content for the same problem reuses the cached evaluation without invoking the AI.

Deterministic Short-Circuit: Asserts that empty, too-short (<100 chars), or prose-only submissions lacking OOP markers are short-circuited with deterministic feedback and never trigger an AI API call.

State Machine Guard: Asserts that a completed submission cannot transition back to `evaluating`.

Evidence Verification Guard: Asserts that exact and normalized quotes pass verification, while fabricated or hallucinated quotes fail verification and have their confidence downgraded to `'low'`.

Full E2E Happy Path (Supertest): Tests the full flow: fetch problems → create attempt 1 → submit → poll result → create attempt 2 → submit revised code → verify per-criterion deltas.

Known Limitations

No Authentication: Uses a hardcoded demo user (`demo-user` or `x-user-id` header). Auth is out of scope for this MVP.

No Diagram Support: Plain text / pseudocode only. Designed deliberately to evaluate domain thinking rather than diagram drawing.

Synchronous Evaluation: Live AI evaluation takes ~10–30 seconds. The frontend polls `/api/submissions/:id` every 2 seconds with an animated status indicator.

No Attempt History Pagination: Returns all past attempts for a problem in a single query; sufficient for a prototype practice session, but would need cursor pagination at scale.

Mock Evaluator Fallback: A local fallback exists for development without API keys, but the live demonstration should be run with a valid `ANTHROPIC_API_KEY`.

Design Decisions Summary

Text Over Diagrams: Text and pseudocode are the most concise medium that provides dense signal on OOP contracts, encapsulation, and class responsibilities without spending engineering budget on a diagram canvas.

Submission Stored Before Evaluation: Submissions are saved to the database as `pending` before the evaluator is invoked. If the external AI service times out or crashes, user work is never lost.

Deterministic / AI Boundary: Code enforces what can be checked deterministically (character counts, keyword tokens, quote presence); the AI is reserved for subjective architectural judgement.

MongoDB Over PostgreSQL: The `Evaluation` record with its embedded, variable `RubricResult` array is naturally document-shaped. MongoDB avoids relational join overhead on every submission fetch.

Synchronous State Machine Over Queue: Following the assignment brief's warning against distributed-systems overreach, a synchronous in-process promise pipeline with polling provides resilience without Redis/BullMQ infrastructure complexity.

Fixed Rubric Over Adaptive Scoring: Consistent rubric dimensions across all attempts are required to compute meaningful per-criterion deltas (e.g. tracking whether *Coupling & Cohesion* improved from attempt 1 to attempt 2).

(For in-depth architectural trade-offs, refer to the Design Note document).

AI Usage Report

This document records key decisions made while working with AI coding tools during the 2-day build of the LLD Practice Platform. It highlights instances where default AI proposals were audited, overridden, simplified, or steered to adhere strictly to the project constraints and grading criteria.

1. AI Suggested a BullMQ/Redis Job Queue — Rejected

What happened: When I asked the AI to design the evaluation flow, it suggested using BullMQ with Redis as a background job queue to process evaluations asynchronously. The reasoning was sound for a high-scale production system — AI evaluation takes 10–30 seconds and blocking the request thread isn't ideal at scale.

What I did: I rejected the queue architecture. The assignment brief explicitly states that "a simple monolith is completely acceptable" and warns against "turning this into a distributed-systems project." For a prototype with a single demo user, an in-process synchronous state machine (`pending` → `evaluating` → `completed` / `failed`) paired with frontend polling is the simplest thing that works. Submissions are saved before evaluation starts, ensuring user data is never lost even if an external call times out.

Lesson: The AI optimized for production scale by default. My role was to enforce the assignment's actual scope — knowing when *not* to build something is critical engineering judgement.

2. AI Suggested a Generic 1–100 AI Score — Overridden

What happened: The first version of the AI evaluator prompt asked Claude to "rate this design out of 100 and provide feedback." It returned a single arbitrary number alongside a paragraph of loose commentary.

What I did: I overrode this with a fixed, multi-dimensional rubric covering specific design dimensions (requirement understanding, class responsibilities, coupling/cohesion, encapsulation, abstraction/patterns, extensibility, edge cases, and explanation quality), each scored 0–5. Crucially, each criterion requires a verbatim evidence quote cited from the learner's text, an identified concern, and an actionable suggestion. The helping guide explicitly warns against "an LLM prompt that simply asks for a 100-point score" and flags it as something evaluators would be less impressed by.

Lesson: The AI's default approach was the exact anti-pattern the assignment warned against. Reviewing the grading criteria beforehand allowed me to redirect the evaluator toward structured, explainable, evidence-backed feedback.

3. AI Dropped the Idempotency Check — Had to Prompt Explicitly

What happened: In the initial implementation of the submission endpoint, the AI generated a clean POST handler that saved the submission and fired the evaluation, but completely omitted duplicate detection. Submitting the identical design twice (via double-click or page retry) created two separate submissions and consumed two redundant AI calls.

What I did: I explicitly prompted for a content hash using Node.js's built-in `crypto` module (SHA-256 of trimmed submission text) stored directly on the `Submission` document. Before invoking the evaluator, the service queries whether a completed evaluation already exists for that `(problemId, contentHash)` pair. If found, it immediately reuses the existing evaluation. This directly satisfied the helping guide's requirement to "avoid duplicate processing where a user retries the same request."

Lesson: The AI generated the standard happy path but overlooked a core operational requirement called out in the brief. I had to know the requirement existed and direct the model to implement it.

4. AI Suggested Matching Against a Reference Solution — Rejected

What happened: When architecting the evaluation approach, the AI recommended storing a canonical "reference solution" for each problem and computing a structural diff or semantic similarity score between the learner's code and the reference.

What I did: I rejected this approach. Low-Level Design problems inherently have multiple valid solutions; a Parking Lot or Vending Machine can be modeled cleanly using different design patterns (e.g. State Pattern vs Strategy Pattern) and different class decompositions. Scoring against a single reference solution penalizes valid alternative architectures. Instead, the prompt evaluates fundamental design qualities (coupling, cohesion, single responsibility, extensibility) independently of any single canonical implementation, respecting the guide's warning to "avoid treating a reference solution as the only correct answer."

Lesson: The AI defaulted to a LeetCode-style test-case/solution matching paradigm. Recognizing that design evaluation requires assessing principles rather than string/AST diffs was an essential engineering distinction.

5. AI Suggested a Separate Rubric Model — Simplified

What happened: The AI initially proposed creating a dedicated `Rubric` collection in MongoDB complete with full CRUD endpoints and database schemas, operating under the assumption that rubrics should be dynamically manageable per problem.

What I did: I simplified the design by treating the rubric as static configuration in code (`rubric.js`) and within the evaluator's system prompt. For a 2-day MVP, rubric dimensions remain constant across problems, with weights defined directly in config. A dedicated MongoDB collection would have added unnecessary database calls, migrations, and schema boilerplate without user-facing value. The helping guide explicitly advises against "design patterns added only to show pattern knowledge."

Lesson: The AI over-engineered the domain model. Knowing the difference between what needs a database lifecycle and what is simply configuration prevented unnecessary architectural bloat.

Summary

The pattern across all these: AI is good at generating structure and happy paths. It's weak at catching edge cases the brief specifically mentions, applying assignment constraints, and knowing when NOT to build something. My job was to read the brief carefully, know the requirements, and override the AI when it optimised for the wrong thing.